Team Notebook

CoU_TernaryTootedTree

COMILLA UNIVERSITY

RAKIBJOY | MESTU | MATRIX10

Last Updated- CUET IUPC

Data Structure

```
//Binary Indexed Tree
template <class DT>
struct BIT
{
    int n;
    vector<DT> fanwick;
    BIT() {}
    BIT(int nn)
    {
        n = nn;
        fanwick.assign(n + 1, 0);// 1-indexed
    }
    DT presum(int i)
    {
        DT an = 0;
        for (; i >= 1; i -= (i & -i))
            an += fanwick[i];
        return an;
    }
    void update(int i, DT val)
    {
        if (i <= 0)
            return;
        for (; i <= n; i += (i & -i))
            fanwick[i] += val;
    }
    void update(int lf, int rt, DT val)
    {
        update(lf, val);
        update(rt + 1, -val);
    }
    DT query(int l, int r)
    {
        return presum(r) - presum(l - 1);
    }
};
//Disjoint Set Union
struct DSU
{
    vector<int>par,sz,rnk;
    DSU(int n)
    {
        par.assign(n+1,0);
        rnk.assign(n+1,0);
        sz.assign(n+1,1);
        for(int i=1; i<=n; i++)
            par[i]=i;
    }
    int get_parent(int n)
    {
        if(par[n]==n)
            return n;
        return par[n]=get_parent(par[n]);
    }
    void union_set(int x,int y)
    {
        x=get_parent(x);
        y=get_parent(y);
        if(x!=y)
        {
            if(rnk[x]>rnk[y])
                swap(x,y);
            par[x]=y;
            if(rnk[x]==rnk[y])
                rnk[y]++;
            sz[x]+=sz[y];
            sz[y]=0;
        }
    }
    int get_size(int n)
    {
        return sz[get_parent(n)];
    }
};
//Mo's Algorithm
int sz;
struct query
{
    int l,r,id;
    inline bool operator<(const query &t)const
    {
        return make_pair(l/sz,r)<make_pair(t.l/sz,t.r);
    }
} q[N];
int n,m,k;
ll v[N];
ll fr[2*N];
ll an=0;
ll ans[N];
void add(int i)
{
    an+=fr[v[i]^k];
    fr[v[i]]++;
}
void del(int i)
{
    fr[v[i]]--;
    an-=fr[v[i]^k];
}
void solve()
{
    cin>>n>>m>>k;
    fr(i,1,n)
    {
        cin>>v[i];
        v[i]^=v[i-1];
    }
```

```
    sz=sqrt(n);
    fr(i,0,m-1)
    {
        int l,r;
        cin>>q[i].l>>q[i].r;
        q[i].l--;
        q[i].id=i;
    }
    sort(q,q+m);
    int l=1,r=0;
    for(int i=0; i<m; i++)
    {
        query Q=q[i];
        while(Q.l<l)
            add(--l);
        while(Q.r>r)
            add(++r);
        while(Q.l>l)
            del(l++);
        while(Q.r<r)
            del(r--);
        ans[Q.id]=an;
    }
    fr(i,0,m-1)
    {
        cout<<ans[i]<<endl;
    }
}
//Segment Tree with Arithmatic progression
int v[N+5];
ll get_sum(ll a,ll n,ll d)
{
    return (((a * 2) + (n - 1) * d) * n) / 2;
}
struct node
{
    ll a,d;
    node operator +(const node &t) const
    {
        return {a+t.a,d+t.d};
    }
};
ll nth(ll a,ll n,ll d)
{
    return a+(n-1)*d;
}
struct Stree
{
#define lf (nd << 1)
#define rt ((nd << 1) | 1)
    long long t[4 * N];
    node lazy[4 * N];
    Stree()
    {
        memset(t, 0, sizeof t);
        memset(lazy, 0, sizeof lazy);
    }
    inline void push(int nd, int lo, int hi)
    {
        if (lazy[nd].a==0&&lazy[nd].d==0)
            return;
        t[nd] = t[nd] + get_sum(lazy[nd].a,hi-lo+1,lazy[nd].d);
        if (lo != hi)
        {
            ll mid=(lo+hi)>>1;
            lazy[lf].a +=nth(lazy[nd].a,lo-lo+1,lazy[nd].d);
            lazy[rt].a +=nth(lazy[nd].a,mid+1-lo+1,lazy[nd].d);
            lazy[lf].d =lazy[nd].d+lazy[lf].d;
            lazy[rt].d =lazy[nd].d+lazy[rt].d;
        }
        lazy[nd] = {0,0};
    }
    inline long long milao(long long x,long long y)
    {
        return x + y;
    }
    inline void uthao(int nd)
    {
        t[nd] = t[lf] + t[rt];
    }
    void build(int nd, int lo, int hi)
    {
        lazy[nd] = {0,0};
        if (lo == hi)
        {
            t[nd] = v[lo];
            return;
        }
        int mid = (lo + hi) >> 1;
        build(lf, lo, mid);
        build(rt, mid + 1, hi);
        uthao(nd);
    }
    void update(int nd, int lo, int hi, int i, int j, long long a,long long d)
    {
        push(nd, lo, hi);
        if (j < lo || hi < i)
            return;
        if (i <= lo && hi <= j)
        {
            lazy[nd] = {nth(a,lo-i+1,d),d}; //set lazy
            push(nd, lo, hi);
            return;
        }
        int mid = (lo + hi) >> 1;
        update(lf, lo, mid, i, j, a,d);
        update(rt, mid + 1, hi, i, j, a,d);
        uthao(nd);
```

```
    }
    long long query(int nd, int lo, int hi, int i, int j)
    {
        push(nd, lo, hi);
        if (i > hi || lo > j)
            return 0;
        if (i <= lo && hi <= j)
            return t[nd];
        int mid = (lo + hi) >> 1;
        return milao(query(lf, lo, mid, i, j), query(rt, mid + 1, hi, i, j));
    }
};
//Segment Tree with Lazy
int v[N];
struct Stree
{
#define lf (nd << 1)
#define rt ((nd << 1) | 1)
    long long t[4 * N], lazy[4 * N];
    Stree()
    {
        memset(t, 0, sizeof t);
        memset(lazy, 0, sizeof lazy);
    }
    inline void push(int nd, int lo, int hi)
    {
        if (lazy[nd] == 0)
            return;
        t[nd] = t[nd] + lazy[nd] * (hi - lo + 1);
        if (lo != hi)
        {
            lazy[lf] = lazy[lf] + lazy[nd];
            lazy[rt] = lazy[rt] + lazy[nd];
        }
        lazy[nd] = 0;
    }
    inline long long milao(long long x,long long y)
    {
        return x + y;
    }
    inline void uthao(int nd)
    {
        t[nd] = t[lf] + t[rt];
    }
    void build(int nd, int lo, int hi)
    {
        lazy[nd] = 0;
        if (lo == hi)
        {
            t[nd] = v[lo];
            return;
        }
        int mid = (lo + hi) >> 1;
        build(lf, lo, mid);
        build(rt, mid + 1, hi);
        uthao(nd);
    }
    void update(int nd, int lo, int hi, int i, int j, long long val)
    {
        push(nd, lo, hi);
        if (j < lo || hi < i)
            return;
        if (i <= lo && hi <= j)
        {
            lazy[nd] = val; //set lazy
            push(nd, lo, hi);
            return;
        }
        int mid = (lo + hi) >> 1;
        update(lf, lo, mid, i, j, val);
        update(rt, mid + 1, hi, i, j, val);
        uthao(nd);
    }
    long long query(int nd, int lo, int hi, int i, int j)
    {
        push(nd, lo, hi);
        if (i > hi || lo > j)
            return 0;
        if (i <= lo && hi <= j)
            return t[nd];
        int mid = (lo + hi) >> 1;
        return milao(query(lf, lo, mid, i, j), query(rt, mid + 1, hi, i, j));
    }
};
//Segment Tree
int v[N];
struct Stree
{
#define lf (nd << 1)
#define rt ((nd << 1) | 1)
    int t[4 * N];
    static const int inf = -1e9;
    Stree()
    {
        memset(t, 0, sizeof t);
    }
    void build(int nd, int lo, int hi)
    {
        if (lo == hi)
        {
            t[nd] = v[lo];
            return;
        }
        int mid = (lo + hi) >> 1;
        build(lf, lo, mid);
        build(rt, mid + 1, hi);
        t[nd] = max(t[lf], t[rt]);
    }
```

```
    void update(int nd, int lo, int hi, int i, int x)
    {
        if (lo > i || hi < i)
            return;
        if (lo == hi && lo == i)
        {
            t[nd] = x;
            return;
        }
        int mid = (lo + hi) >> 1;
        update(lf, lo, mid, i, x);
        update(rt, mid + 1, hi, i, x);
        t[nd] = max(t[lf], t[rt]);
    }
    int query(int nd, int lo, int hi, int i, int j)
    {
        if (lo > j || hi < i)
            return inf;
        if (lo >= i && hi <= j)
            return t[nd];
        int mid = (lo + hi) >> 1;
        int L = query(lf, lo, mid, i, j);
        int R = query(rt, mid + 1, hi, i, j);
        return max(L, R);
    }
};
//Sparse Table
int v[N+5];
int sptab[N+5][26];
int Log[N+5];
void LogPreCalc()
{
    Log[1]=0;
    for(int i=2; i<=N; i++)
        Log[i]=Log[i/2]+1;
}
void SPgen(int n)
{
    for(int i=1; i<=n; i++)
        sptab[i][0]=v[i];
    for(int j=1; (1<<j)<=n; j++)
    {
        for(int i=1; i+(1<<j)-1<=n; i++)
        {
            sptab[i][j]=min(sptab[i][j-1],sptab[i+(1<<(j-1))][j-1]);
        }
    }
}
int RmnQ(int lo,int hi)
{
    int x=Log[hi-lo+1];
    return min(sptab[lo][x],sptab[hi-(1<<x)+1][x]);
}
long long RngSum(int lo,int hi,int n)
{
    long long ans=0;
    for(int j=(int)log2(n*1.0)+1 ; j>=0 ; j--)
    {
        if((1<<j) <= hi-lo+1)
        {
            ans += sptab[lo][j];
            lo += (1<<j);
        }
    }
    return ans;
}
//Monotonic Queue
struct monotonic_queue
{
    deque<pair<int,int> >dq;
    void add(int val,int in)
    {
        while(!dq.empty()&&dq.back().first>=val) //strictly increasing order
            dq.pop_back();
        dq.push_back({val,in});
    }
    void del(int in)
    {
        if(!dq.empty() && dq.front().second==in)
            dq.pop_front();
    }
};
//Trie Tree
struct Trie
{
    struct node
    {
        node* nxt[26];
        bool endmark;
        node()
        {
            for(int i=0; i<26; i++)
                nxt[i]=NULL;
            endmark=false;
        }
    }*root;
    Trie()
    {
        root = new node();
    }
    void Add(string &s)
    {
        node* cur = root;
        int sz=s.size();
        for(int i=0; i<sz; i++)
        {
            int c=s[i]-'a';
            if(cur->nxt[c]== NULL)
```

```
                cur->nxt[c] = new node();
            cur=cur->nxt[c];
        }
        cur->endmark=true;
    }
    int query(string &s)
    {
        node* cur = root;
        int sz=s.size();
        for(int i=0; i<sz; i++)
        {
            int c=s[i]-'a';
            if(cur->nxt[c]== NULL)
                return false;
            cur=cur->nxt[c];
        }
        return cur->endmark;
    }
    void del(node* cur)
    {
        for (int i = 0; i < 26; i++)
            if (cur -> nxt[i])
                del(cur -> nxt[i]);
        delete(cur);
    }
};
//Merge Sort Tree
class Merge_Sort_Tree{
    public:
    vector<int> v;
    vector<vector<int>> tree;
    int n;
    Merge_Sort_Tree(int _n){
        n=_n;
        v.resize(n+1);
        tree.resize(4*n);
    }
    void Merge(int nd){
        int l = nd*2,r=nd*2+1;
        int n1 = tree[nd*2].size();
        int n2 = tree[nd*2+1].size();
        int i=0,j=0;
        while(i<n1&&j<n2){
            if(tree[l][i]<tree[r][j]){
                tree[nd].push_back(tree[l][i++]);
            }
            else{
                tree[nd].push_back(tree[r][j++]);
            }
        }
        while(i<n1)tree[nd].push_back(tree[l][i++]);
        while(j<n2)tree[nd].push_back(tree[r][j++]);
    }
    void build(int nd, int b, int e){
        if(b>e)return;
        if(b==e){
            tree[nd].push_back(v[b]);
            return ;
        }
        int m = (b+e)/2;
        build(nd*2,b,m);
        build(nd*2+1,m+1,e);
        Merge(nd);
    }
    int query(int nd, int b, int e, int l, int r, int k){
        if(b>r||e<l)return 0;
        if(l<=b&&e<=r){
            int pos = upper_bound(tree[nd].begin(),tree[nd].end(),k)-
tree[nd].begin();
            return tree[nd].size()-pos;
        }
        int m=(b+e)/2;
        int p1 = query(nd*2,b,m,l,r,k);
        int p2 = query(nd*2+1,m+1,e,l,r,k);
        return p1+p2;
    }
};
//SQRT Decomposition
vector<long long> vcr;
vector<vector<long long >> blocks;
long long N, block_size;
void initialize()
{
    block_size = sqrt(N);
    long long block_no = -1;
    for (int i = 0; i < N; ++i)
    {
        if (i % block_size == 0)
        {
            block_no++;
            vector<long long> s;
            blocks.emplace_back(s);
        }
        blocks[block_no].push_back(vcr[i]);
    }
    for (auto &i: blocks)
    {
        sort(i.begin(), i.end());
    }
}
void query(int l, int r, long long v, int p, long long u)
{
    long long k = 0;
    int l1 = l;
    while (l % block_size && l <= r)
    {
        if (vcr[l] < v)
            k++;
```

```
        l++;
    }
    while (l + block_size <= r)
    {
        int sz=l / block_size;
        k += lower_bound(blocks[sz].begin(), blocks[sz].end(), v) -
             blocks[sz].begin();
        l += block_size;
    }
    while (l <= r)
    {
        if (vcr[l] < v)
            k++;
        l++;
    }
    int sz=p / block_size;
    int x = lower_bound(blocks[sz].begin(), blocks[sz].end(), vcr[p])-
            blocks[sz].begin();
    blocks[sz][x] = (u * k) / (r - l1 + 1);
    vcr[p] = (u * k) / (r - l1 + 1);
    sort(blocks[sz].begin(), blocks[sz].end());
}
void print_array()
{
    for (auto &i: vcr)
    {
        cout << i << endl;
    }
}
void sqrt_Decomposition(vector<long long> &vc)
{
    N = vc.size();
    vcr = vc;
    initialize();
}
// Gnu Pubds
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<typename T> using ordered_set =tree<T, null_type,less<int>,
rb_tree_tag,tree_order_statistics_node_update>;
//order_of_key (k) : Number of items strictly smaller than k .
//find_by_order(k) : K-th element in a set (counting from zero).
template<typename T> using ordered_multiset = tree<T, null_type,less_equal<T>,
rb_tree_tag,tree_order_statistics_node_update>;
Erase -
if(os.upper_bound(x)!=os.end())
    os.erase(os.upper_bound(x));
For Pair- typedef tree<
pair<int, int>,
     null_type,
     less<pair<int, int>>,
     rb_tree_tag,
     tree_order_statistics_node_update> ordered_set;
```

```
//Custom Hash FOR pair
struct custom_hash
{
    size_t operator()(const pair<int,int>&x)const
    {
        return ((long long)x.first)^(((long long)x.second)<<32);
    }
};
// gp_hash_table is effitient
```

Graph Theory

```
//Bipartite Matching
vector<vector<int>> graph;
vector<int> match, dist;

bool bfs() {
    int i, u, v;
    queue<int> Q;
    for (i = 1; i < lim; i++) {
        if (match[i] == 0) {
            dist[i] = 0;
            Q.push(i);
        } else dist[i] = INT_MAX;
    }
    dist[0] = INT_MAX;
    while (!Q.empty()) {
        u = Q.front();
        Q.pop();
        if (u != 0) {
            for (i = 0; i < graph[u].size(); i++) {
                v = graph[u][i];
                if (dist[match[v]] == INT_MAX) {
                    dist[match[v]] = dist[u] + 1;
                    Q.push(match[v]);
                }
            }
        }
    }
    return (dist[0] != INT_MAX);
}

bool dfs(int u) {
    int i, v;
    if (u != 0) {
        for (i = 0; i < graph[u].size(); i++) {
            v = graph[u][i];
            if (dist[match[v]] == dist[u] + 1) {
                if (dfs(match[v])) {
                    match[v] = u;
                    match[u] = v;
                    return true;
                }
            }
        }
        dist[u] = INT_MAX;
```

```
        return false;
    }
    return true;
}

int hopcroft_karp() {
    int matching = 0, i;
    while (bfs())
        for (i = 1; i < match.size(); i++)
            if (match[i] == 0 && dfs(i))
                matching++;
    return matching;
}
//Centroid of a Tree
int get_cen(int n,int par)
{
    for(auto it:adj[n])
    {
        if(it==par)
            continue;
        if(sz[it]>size_of_tree/2)
            return get_cen(it,n);
    }

    return n;
}
//Strongly Connected Components
class SCC{
  public:
  vector<int>order,component;
  vector<bool> used;
  int n,rt;
  vector<vector<int>>adj,adj_rev;
  vi root;
  SCC(int _n){
      n = _n;
      rt=0;
      root.resize(n+1);
      used = vector<bool>(n+1);
      adj = adj_rev = vector<vector<int>>(n+1);
  }
  void readGraph(int m){
      for(int i=0; i<m; i++){
          int a,b;
          cin>>a>>b;
          adj[a].push_back(b);
          adj_rev[b].push_back(a);
      }
  }
  void dfs1(int nd){
      used[nd] = true;
      for(auto v:adj[nd]){
          if(!used[v]){
              dfs1(v);
          }
      }
      order.push_back(nd);
  }
  void dfs2(int nd){
      root[nd] = rt;
      used[nd] = true;
      for(auto v:adj_rev[nd]){
          if(!used[v]){
              dfs2(v);
          }
      }
  }
  void call_all_function(){
      for(int i=1; i<=n; i++){
          if(!used[i]){
              dfs1(i);
          }
      }
      used = vector<bool>(n+1,false);
      reverse(all(order));
      for(auto i:order){
          if(!used[i]){
              ++rt;
              dfs2(i);
          }
      }
  }
};
//Minimum Spanning Tree
//Need DSU here
struct edge
{
    int u, v, w;
    bool operator<(const edge& cm) const
    {
        return w < cm.w;
    }
};
vector<edge> e;
ll mst(int n)
{
    sort(e.begin(), e.end());
    DSU d(n);
    ll cnt = 0, s = 0;
    for (int i = 0; i < (int)e.size(); i++)
    {
        int u=e[i].u,v=e[i].v;
        if (d.get_parent(u) != d.get_parent(v))
        {
            d.union_set(u,v);
            cnt++;
            s += (ll)e[i].w;
            if (cnt == n - 1)
```

```
                break;
            }
        }
    }
    return s;
}
//Topological Sort with Cycle Detection
vector<int> v;
vector<int> adj[27];
int col[27];
bool isCycle=0;
void cycle_detection(int n)
{
    if(col[n])
    {
        if(col[n]==1)
            isCycle=1;
        return;
    }
    col[n]=1;
    for(auto it:adj[n])
        cycle_detection(it);
    col[n]=2;
    v.push_back(n);
}
//Articuation Bridge
class Find_bridge{
    public:
    int n,Time,m;
    vector<vector<int>>adj;
    vector<bool>visit;
    vector<int>tim,low;
    vector<pair<int,int>> bridges;
    Find_bridge(int _n):n(_n){
        Time=0;
        adj.resize(n+1);
        visit.resize(n+1);
        tim.resize(n+1);
        low.resize(n+1,INT_MAX);
    }
    void readGraph(){
        int a,b;
        cin>>m;
        for(int i=0; i<m; i++){
            cin>>a>>b;
            adj[a].push_back(b);
            adj[b].push_back(a);
        }
    }
    void dfs(int node, int par){
        tim[node]=low[node]=++Time;
        visit[node]=1;
        for(auto to:adj[node]){
            if(par==to)continue;
            if(visit[to]){
                low[node]=min(low[node],tim[to]);
            }
            else{
                dfs(to,node);
                low[node] = min(low[node],low[to]);
                if(low[to]>tim[node]){
                    bridges.push_back({min(to,node),max(to,node)});
                }
            }
        }
    }
};
//Articulation Point
class ArticulationPoint{
    public:
    int n;
    vector<vector<int>> adj;

    vector<bool> visited,artPoint;
    vector<int> tin, low;
    int timer;
    ArticulationPoint(int _n){
        n = _n;
        timer = 0;
        visited.assign(n+1, false);
        tin.assign(n+1, -1);
        low.assign(n+1, -1);
        artPoint.assign(n+1,false);
        adj.resize(n+1);
    }
    void readGraph(int m){
        for(int i=0; i<m; i++){
            int a,b;
            cin>>a>>b;
            adj[a].push_back(b);
            adj[b].push_back(a);
        }
    }
    void dfs(int v, int p = -1) {
        visited[v] = true;
        tin[v] = low[v] = timer++;
        int children=0;
        for (int to : adj[v]) {
            if (to == p) continue;
            if (visited[to]) {
                low[v] = min(low[v], tin[to]);
            } else {
                dfs(to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= tin[v] && p!=-1){
                    artPoint[v]=true;
                }
                ++children;
            }
```

```
        }
        if(p == -1 && children > 1)
            artPoint[v]=true;
    }

    int find_cutpoints() {
        int cnt=0;
        for (int i = 1; i <= n; ++i) {
            if (!visited[i])
                dfs (i);
            cnt += artPoint[i];
        }
        return cnt;
    }
};
//Lowest Common Ancestor
const int Log=15;
class Lca{
    public:
    vector<vector<int>> up,g;
    vector<int> depth;
    int n;
    Lca(int _n){
        n = _n;
        up.resize(n+1,vector<int>(Log,-1));
        g.resize(n+1);
        depth.resize(n+1);
    }
    void readTree(int root=1){
        int a,b;
        for(int i=1; i<n; i++){
            cin>>a>>b;
            g[a].push_back(b);
            g[b].push_back(a);
        }
        dfs(root,root,0);
        cal_sparse_table();
    }
    void dfs(int nd, int par, int d){
        depth[nd]=d,up[nd][0]=par;
        for(auto to:g[nd]){
            if(to==par)continue;
            dfs(to,nd,d+1);
        }
    }
    void cal_sparse_table(){
        for(int i=1; i<Log; i++){
            for(int j=1; j<=n; j++){
                if(up[j][i-1]!=-1)
                    up[j][i] = up[up[j][i-1]][i-1];
            }
        }
    }
    int lca_query(int u, int v){
        if(depth[u] < depth[v]) swap(u,v);
        long long diff = depth[u] - depth[v];
        for(long long i = 0; i < Log; i++) if( (diff>>i)&1 ) u = up[u][i];
        if(u == v) return u;
        for(long long i = Log-1; i >= 0; i--) {
            if(up[u][i] != up[v][i]) {
                    u = up[u][i];
                    v = up[v][i];
            }
        }
        return up[u][0];
    }
   int x_thParent(int u, int x){
        for(long long i = 0; i < Log; i++) if( (x>>i)&1 ) u = up[u][i];
        return u;
   }
   int cal_dis(int a, int b, int l){
        return depth[a]+depth[b] - depth[l]*2;
   }
};
//Tree Diameter
class TreeDiameter{
public:
    int n,leaf,diameter;
    pair<int,int>diameterNodes,center;
    vector<int>dis,parent;
    vector<vector<int>>adj;
    TreeDiameter(int _n){
        n = _n;
        diameter = 0;
        leaf = -1;
        adj.resize(n+1);
        dis.resize(n+1,INT_MAX);
        parent.resize(n+1);
    }
    void readTree(){
        for(int i=1; i<n; i++){
            int a,b;
            cin>>a>>b;
            adj[a].push_back(b);
            adj[b].push_back(a);
            if(adj[a].size()==1)leaf = a;
            if(adj[b].size()==1)leaf = b;
        }
    }
    void dfs(int node){
        for(auto to:adj[node]){
            if(dis[to] > dis[node]+1){
                dis[to] = dis[node]+1;
                parent[to] = node;
                dfs(to);
            }
        }
    }
```

```
    void getMaxDisNode(int &a){
        diameter = 0;
        for(int i=1; i<=n; i++){
            if(diameter < dis[i]){
                diameter = dis[i];
                a = i;
            }
        }
    }
    void findDiemater(){
        if(leaf==-1)return;
        dis[leaf] = 0;
        parent[leaf] = leaf;
        dfs(leaf);
        getMaxDisNode(diameterNodes.first);
        dis.assign(n+1,INT_MAX);
        parent[diameterNodes.first] = diameterNodes.first;
        dis[diameterNodes.first]=0;
        dfs(diameterNodes.first);
        getMaxDisNode(diameterNodes.second);
    }
    void findCenter(){
        if(diameter&1){
            int radius = diameter/2 +1;
            int cur = diameterNodes.second;
            while(dis[cur]!=radius){
                cur = parent[cur];
            }
            center = {cur,parent[cur]};
        }
        else{
            int radius = diameter/2;
            int cur = diameterNodes.second;
            while(dis[cur]!=radius){
                cur = parent[cur];
            }
            center = {cur,cur};
        }
    }
};
//Bellman Ford
class BellmanFord{
  public:
  int n,m;
  vector<pair<int,pair<int,int>>> edges;
  vector<int>parent;
  vector<long long>dis;
  BellmanFord(int _n, int _m){
      n = _n;
      m = _m;
      parent.resize(n+1);
      dis.assign(n+1,1LL<<62);
  }
  void readGraph(){
      for(int i=0; i<m; i++){
          int a,b,c;
          cin>>a>>b>>c;
          edges.pb({c,{a,b}});
      }
  }
  vi findNegCycle(){
      int lastNode=-1;
      dis[1]=0;
      for(int i=0; i<=n; i++){
          lastNode = -1;
          for(auto x:edges){
              int a=x.second.first;
              int b=x.second.second;
              ll  c=x.first;
              if(dis[b]>dis[a]+c){
                  dis[b]=dis[a]+c;
                  lastNode=b;
                  parent[b]=a;
              }
          }
      }
      vi res;
      if(lastNode==-1)return res;
      for(int i=0; i<n; i++)lastNode=parent[lastNode];
      for(int y=lastNode; ;y=parent[y]){
          res.pb(y);
          if(y==lastNode and res.size()>1)break;
      }
      return res;
  }

};
//Centroid Decomposition
class CentroidDecomposition{
public:
    vector<vector<int>>adj,lightAdj;
    vector<bool>removed;
    vector<int>subtree,parent;
    int n,root;
    CentroidDecomposition(int _n){
        n = _n;
        root=1;
        adj.resize(n+1);
        lightAdj.resize(n+1);
        removed.resize(n+1);
        subtree.resize(n+1);
        parent.resize(n+1);
    }
    void readTree(){
        for(int i=1; i<n; i++){
            int a,b;
            cin>>a>>b;
            adj[a].push_back(b);
```

```
            adj[b].push_back(a);
        }
    }
    int get_subtree_size(int node, int par=-1){
        subtree[node]=1;
        for(auto x:adj[node]){
            if(!removed[x] and par!=x){
                subtree[node] += get_subtree_size(x,node);
            }
        }
        return subtree[node];
    }
    int get_centroid(int node, int desired, int par=-1){
        for(auto x:adj[node]){
            if(!removed[x] and x!=par and desired<=subtree[x]){
                return get_centroid(x,desired,node);
            }
        }
        return node;
    }
    void centroid_decomposition(int node, int par=-1){
        int centroid = get_centroid(node,get_subtree_size(node)>>1);
        removed[centroid]=true;
        if(par!=-1){
            lightAdj[par].push_back(centroid);
            parent[centroid]=par;
        }
        else root=centroid;
        for(auto x:adj[centroid]){
            if(!removed[x])
                centroid_decomposition(x,centroid);
        }
    }
```

```
//Max Flow
const int sz = 105;
struct Edge
{
    int u,v,cap;
    Edge() {}
    Edge(int uu,int vv,int _cap)
    {
        u = uu;
        v = vv;
        cap = _cap;
    }
};
vector<Edge>edge;
vector<int>edgeno[sz];
bool vis[sz];
int src,sink;
void AddEdge(int u,int v,int w)
{
    edge.push_back(Edge(u, v, w));
    edge.push_back(Edge(v, u, 0));///As it's an unidirectional edge

    edgeno[u].push_back(edge.size() - 2);  ///Even indexed
    edgeno[v].push_back(edge.size() - 1);  ///Odd indexed
}
int DFS(int u,int flow=1e9)
{
    vis[u] = 1;
    if(u == sink)
        return flow;
    int an = 0;
    for(int i=0 ; i<(int)edgeno[u].size() ; i++)
    {
        int id = edgeno[u][i];
        Edge &cr = edge[id];

        if(!cr.cap || vis[cr.v])
            continue;

        an = DFS(cr.v, min(flow, cr.cap));

        if(an)
        {
            Edge &rev = edge[id ^ 1];  ///Every forward edge stored in even
index & every backward edge stored in next odd index
            cr.cap -= an;
            rev.cap += an;
            return an;
        }
    }
    return an;
}
int FordFulkerson()
{
    int MaxFlow=0;
    while(1)
    {
        memset(vis, 0, sizeof(vis));
        int possible = DFS(src);
        if(possible == 0)
            break;
        MaxFlow += possible;
    }
    return MaxFlow;
}
//Min Cost Max Flow
//Works for both directed, undirected and with negative cost too
//doesn't work for negative cycles
//for undirected edges just make the directed flag false
//Complexity: O(min(E^2 *V log V, E logV * flow))
using T = long long;
const T inf = 1LL << 61;
struct MCMF {
  struct edge {
```

```
    int u, v;
    T cap, cost;
    int id;
    edge(int _u, int _v, T _cap, T _cost, int _id) {
      u = _u;
      v = _v;
      cap = _cap;
      cost = _cost;
      id = _id;
    }
  };
  int n, s, t, mxid;
  T flow, cost;
  vector<vector<int>> g;
  vector<edge> e;
  vector<T> d, potential, flow_through;
  vector<int> par;
  bool neg;
  MCMF() {}
  MCMF(int _n) { // 0-based indexing
    n = _n + 10;
    g.assign(n, vector<int> ());
    neg = false;
    mxid = 0;
  }
  void add_edge(int u, int v, T cap, T cost, int id = -1, bool directed = true)
{
    if(cost < 0) neg = true;
    g[u].push_back(e.size());
    e.push_back(edge(u, v, cap, cost, id));
    g[v].push_back(e.size());
    e.push_back(edge(v, u, 0, -cost, -1));
    mxid = max(mxid, id);
    if(!directed) add_edge(v, u, cap, cost, -1, true);
  }
  bool dijkstra() {
    par.assign(n, -1);
    d.assign(n, inf);
    priority_queue<pair<T, T>, vector<pair<T, T>>, greater<pair<T, T>> > q;
    d[s] = 0;
    q.push(pair<T, T>(0, s));
    while (!q.empty()) {
      int u = q.top().second;
      T nw = q.top().first;
      q.pop();
      if(nw != d[u]) continue;
      for (int i = 0; i < (int)g[u].size(); i++) {
        int id = g[u][i];
        int v = e[id].v;
        T cap = e[id].cap;
        T w = e[id].cost + potential[u] - potential[v];
        if (d[u] + w < d[v] && cap > 0) {
          d[v] = d[u] + w;
          par[v] = id;
          q.push(pair<T, T>(d[v], v));
        }
      }
    }
    for (int i = 0; i < n; i++) { // update potential
      if(d[i] < inf) potential[i] += d[i];
    }
    return d[t] != inf;
  }
  T send_flow(int v, T cur) {
    if(par[v] == -1) return cur;
    int id = par[v];
    int u = e[id].u;
    T w = e[id].cost;
    T f = send_flow(u, min(cur, e[id].cap));
    cost += f * w;
    e[id].cap -= f;
    e[id ^ 1].cap += f;
    return f;
  }
  //returns {maxflow, mincost}
  pair<T, T> solve(int _s, int _t, T goal = inf) {
    s = _s;
    t = _t;
    flow = 0, cost = 0;
    potential.assign(n, 0);
    if (neg) {
      // run Bellman-Ford to find starting potential
      d.assign(n, inf);
      for (int i = 0, relax = true; i < n && relax; i++) {
        for (int u = 0; u < n; u++) {
          for (int k = 0; k < (int)g[u].size(); k++) {
            int id = g[u][k];
            int v = e[id].v;
            T cap = e[id].cap, w = e[id].cost;
            if (d[v] > d[u] + w && cap > 0) {
              d[v] = d[u] + w;
              relax = true;
            }
          }
        }
      }
      for(int i = 0; i < n; i++) if(d[i] < inf) potential[i] = d[i];
    }
    while (flow < goal && dijkstra()) flow += send_flow(t, goal - flow);
    flow_through.assign(mxid + 10, 0);
    for (int u = 0; u < n; u++) {
      for (auto v : g[u]) {
        if (e[v].id >= 0) flow_through[e[v].id] = e[v ^ 1].cap;
      }
    }
    return make_pair(flow, cost);
  }
};
```

```
//DSU on Tree
///counting the max sum of max frequent color
///for every subtree
int a;
vi adj[N];
int sz[N],col[N],cnt[N];
vi nodes[N];
long long mxcol=0,sum[N],ans[N];
void count_size(int n,int p)
{
    sz[n]=1;
    for(auto it:adj[n])
    {
        if(it==p)
            continue;
        count_size(it,n);
        sz[n]+=sz[it];
    }
}
void dsu(int n,int p,bool keep)
{
    int Max = -1,bigchild = -1;
    for(auto it:adj[n])
    {
        if(it!=p&&sz[it]>Max)
        {
            Max=sz[it];
            bigchild=it;
        }
    }
    for(auto it:adj[n])
    {
        if(it!=p&&it!=bigchild)
        {
            dsu(it,n,0);
        }
    }
    if(bigchild!=-1)
    {
        dsu(bigchild,n,1);
        swap(nodes[n],nodes[bigchild]);
    }
    nodes[n].push_back(n);
    cnt[col[n]]++;
    sum[cnt[col[n]]]+=col[n];
    if(cnt[col[n]]>mxcol)
    {
        mxcol++;
    }
    for(auto u:adj[n])
    {
        if(u==p||u==bigchild)
            continue;
        for(auto it:nodes[u])
        {
            nodes[n].push_back(it);
            cnt[col[it]]++;
            sum[cnt[col[it]]]+=col[it];
            if(cnt[col[it]]>mxcol)
            {
                mxcol++;
            }
        }
    }
    ans[n]=sum[mxcol];
    if(keep==0)
    {
        for(auto it:nodes[n])
        {
            sum[cnt[col[it]]]-=col[it];
            cnt[col[it]]--;
            if(sum[mxcol]==0)
            {
                mxcol--;
            }
        }
    }

}
```

Number Theory

```
//Discrete Logarithm
// Returns minimum x for which a ^ x % m = b % m, a and m are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}
//Linear Diophantina
int gcd(int a, int b, int& x, int& y) {
```

```
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

$$x = x_0 + k * \frac{b}{g}$$
$$y = y_0 + k * \frac{a}{g}$$

```
//Extended Euclid and Mod inverse
ll extended_euclid(ll a, ll b, ll &x, ll &y)
{
        if (b == 0)
        {
                x = 1, y = 0;
                return a;
        }
        ll xx, yy;
        ll g = extended_euclid(b, a % b, xx, yy);
        x = yy;
        y = xx - yy * (a / b);
        return g;
}
ll mod_inverse(ll a, ll m)
{
        ll x, y;
        ll gc = extended_euclid(a, m, x, y);
        if (gc != 1) return -1;
        return (x % m + m) % m;
}
//fact,bigmod,ncr,phi,com
ll fact[N+5];
void pre()
{
    fact[0]=1;
    fr(i,1,N)
    {
        fact[i]=i*fact[i-1];
        fact[i]%=MOD;
    }
}
int BigMod(long long p, long long q, const int mod)
{
    int ans = 1 % mod;
    p %= mod;
    if (p < 0)
        p += mod;
    while (q)
    {
        if (q & 1)
            ans = (long long) ans * p % mod;
        p = (long long) p * p % mod;
        q >>= 1;
    }
    return ans;
}
ll ncr(ll n,ll r)
{
    if(n<r)
        return 0;
    ll an=1;
    an*=fact[n];
    ll d=fact[r];
    d*=fact[n-r];
    d%=MOD;
    an*=BigMod(d,MOD-2,MOD);
    an%=MOD;
    return an;
}
ll phi[N+5];
void euler()
{
    for(ll i=1; i<=N; i++)
        phi[i]=i;
    for(ll i=2; i<=N; i++)
    {
        if(phi[i]==i)
        {
            for(ll j=i; j<=N; j+=i)
            {
                phi[j]/=i;
                phi[j]*=(i-1);
            }
        }

    }
}
int Totient(int num)
{
        int ans = num;
        for(int i = 2; i * i <= num; i++)
        {
```

```
                if(num % i)
                        continue;
                while(num % i == 0)
                        num /= i;
                ans -= ans / i;
        }
        if(num > 1)
                ans -= ans / num;
        return ans;
}
ll com[1001][1001];
void combination()
{
    com[1][1]=1;
    com[0][0]=1;
    for(ll i=1; i<=1000; i++)
    {
        com[i][0]=1;
        for(ll j=1; j<=i; j++)
        {
            com[i][j]=com[i-1][j]+com[i-1][j-1];
            com[i][j]%=mod;
        }
    }
}
//Sieve
bool prime[N+5];
void primenire()
{
    int i, j;
    prime[0]=true;
    prime[1]=true;
    for(i=4; i<=N; i+=2)
    {
        prime[i]=true;
    }
    for(i=3; i*i<=N; i+=2)
    {
        if(!prime[i])
        {
            for(j=i*i; j<=N; j+=2*i)
            {
                prime[j]=true;
            }
        }
    }
}
//Sum of Divisor
long long SumOfDivisor(long long n)
{
    long long ans=1;
    for(long long i=0; prime[i]*prime[i]<=n; i++)
    {
        long long sum=0,p=1;
        while(n%prime[i]==0)
        {
            n/=prime[i];
            p*=prime[i];
            sum+=p;
        }
        ans*=(sum+1);
    }
    if(n>1)
        ans*=(n+1);
    return ans;
}
//Miller Robin
long long bigmod(unsigned long long n, unsigned long long p, unsigned long long
m)
{

    unsigned long long x = 1;
    n %= m;
    while (p)
    {
        if (p & 1)
            x = (__uint128_t) x * n % m;
        n = (__uint128_t) n * n % m;
        p >>= 1;
    }
    return x;
}
bool isComposite(unsigned long long n, int p, unsigned long long d, unsigned
long long m)
{
    unsigned long long x = bigmod(n, d, m);
    if (x == 1 || x == m - 1)
        return false;
    for (int i = 1; i < p; ++i)
    {
        x = (__uint128_t) x * x % m;
        if (x == m - 1)
            return false;
    }
    return true;
}
bool isPrime(unsigned long long n)
{
    if (n < 2)
        return false;
    unsigned long long m = n;
    n--;
    int p = 0;
    while (n % 2 == 0)
        n >>= 1, p++;
    for (auto &i:
            {
                2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
```

```
            })
    {
        if (m == i)
            return true;
        if (isComposite(i, p, n, m))
            return false;
    }
    return true;
}
//Number of Divisor
long long NumberOfDivisor(long long n)
{
    long long ans=1;
    for(long long i=0; prime[i]*prime[i]<=n; i++)
    {
        long long counter=0;
        while(n%prime[i]==0)
        {
            n/=prime[i];
            counter++;
        }
        ans*=(counter+1);
    }
    if(n>1)
        ans*=2;
    return ans;
}
```

String

```
//KMP
template<typename T>
class kmp
{
    vector<int> indx;
public:
    void lps(T &patt)
    {
        indx.resize(patt.size(), 0);
        int i = 0, j = 1;
        while (j < patt.size())
        {
            if (patt[i] == patt[j])
                indx[j] = ++i, j++;
            else
            {
                if (i != 0)
                    i = indx[i - 1];
                else
                    indx[j] = 0, j++;
            }
        }
    }

    bool match(T &text, T &patt)
    {
        int i = 0, j = 0;
        while (j < text.size())
        {
            if (patt[i] == text[j])
                i++, j++;
            else
            {
                if (i != 0)
                    i = indx[i - 1];
                else
                    j++;
            }
            if (i == patt.size())
                return true;
        }
        return false;
    }

    int frequency(T &text, T &patt)
    {
        int i = 0, j = 0, cnt = 0;
        while (j < text.size())
        {
            if (patt[i] == text[j])
                i++, j++;
            else
            {
                if (i != 0)
                    i = indx[i - 1];
                else
                    j++;
            }
            if (i == patt.size())
                cnt++, i = indx[i - 1];
        }
        return cnt;
    }
};
//Z-Algo
vector<int> z_maker(string &s)
{
    int sz=s.size();
    vector<int>zv(sz);
    int l=0,r=0;
    for(int i=1; i<sz; i++)
    {
        if(i<=r)
            zv[i]=min(zv[i-l],r-i+1);
        while(i+zv[i]<sz&&s[zv[i]]==s[i+zv[i]])
            zv[i]++;
        if(i+zv[i]-1>r)
            l=i,r=i+zv[i]-1;
    }
    return zv;
```

```
}
//Hashing
//Need BigMod function here
const int MOD1 = 127657753, MOD2 = 987654319;
const int b1 = 141, b2 = 277;
int inp1, inp2;
pair<int, int> pw[N], inpw[N];
void precalc()
{
    pw[0] =  {1, 1};
    for (int i = 1; i < N; i++)
    {
        pw[i].first = 1LL * pw[i - 1].first * b1 % MOD1;
        pw[i].second = 1LL * pw[i - 1].second * b2 % MOD2;
    }
    inp2 = BigMod(b2, MOD2 - 2, MOD2);
    inp1 = BigMod(b1, MOD1 - 2, MOD1);
    inpw[0] =  {1, 1};
    for (int i = 1; i < N; i++)
    {
        inpw[i].first = 1LL * inpw[i - 1].first * inp1 % MOD1;
        inpw[i].second = 1LL * inpw[i - 1].second * inp2 % MOD2;
    }

}
struct Hashing
{
    int sz;
    string s; // 0 - indexed string
    vector<pair<int, int>> h_tab; // 1 - indexed vector
    Hashing() {}
    Hashing(string _ss)
    {
        sz = _ss.size();
        s = _ss;
        h_tab.emplace_back(0, 0);
        for (int i = 0; i < sz; i++)
        {
            pair<int, int> p;
            p.first = (h_tab[i].first + 1LL * pw[i].first * s[i] % MOD1) % MOD1;
            p.second = (h_tab[i].second + 1LL * pw[i].second * s[i] % MOD2) %
MOD2;
            h_tab.push_back(p);
        }
    }
    pair<int, int> get_hash_value(int lo, int hi)    // 1 - indexed
    {
        assert(1 <= lo && lo <= hi && hi <= sz);
        pair<int, int> ans;
        ans.first = (h_tab[hi].first - h_tab[lo - 1].first + MOD1) * 1LL *
inpw[lo - 1].first % MOD1;
        ans.second = (h_tab[hi].second - h_tab[lo - 1].second + MOD2) * 1LL *
inpw[lo - 1].second % MOD2;
        return ans;
    }
    pair<int, int> get_hash_value()
    {
        return get_hash_value(1, sz);
    }
};
```

```
pair<int,int> mer(pii x,pii y,int z)
{
    pii an;
    an.F=((1LL*x.F*pw[z].F)%MOD1+y.F)%MOD1;
    an.S=((1LL*x.S*pw[z].S)%MOD2+y.S)%MOD2;
    return an;
}
```

```
//Suffix Array
struct suf_array
{
    string s;
    int sz;
    vector<int>st,rnk,lcp;
    suf_array() {}
    suf_array(string &ss)
    {
        s=ss;
        s+='$';
        sz=s.size();
        st.resize(sz);
        rnk.resize(sz);
        lcp.resize(sz);
    }
    void count_sort()
    {
        vector<int>cnt(sz);
        for(auto x:rnk)
            cnt[x]++;
        vector<int>st_new(sz);
        vector<int>pos(sz);
        pos[0]=0;
        for(int i=1; i<sz; i++)
            pos[i]=pos[i-1]+cnt[i-1];
        for(auto x:st)
        {
            int i=rnk[x];
            st_new[pos[i]]=x;
            pos[i]++;
        }
        st=st_new;
    }
    void build()
    {
        {
            ///for k=0;
            vector<pair<char,int>>tm(sz);
```

```
        for(int i=0; i<sz; i++)
            tm[i]= {s[i],i};
        sort(tm.begin(),tm.end());
        for(int i=0; i<sz; i++)
            st[i]=tm[i].second;
        rnk[st[0]]=0;
        for(int i=1; i<sz; i++)
        {
            if(tm[i].first==tm[i-1].first)
                rnk[st[i]]=rnk[st[i-1]];
            else
                rnk[st[i]]=rnk[st[i-1]]+1;

        }
    }
    int k=1;
    while(k<sz)
    {
        for(int i=0; i<sz; i++)
            st[i]=(st[i]-k+sz)%sz;
        count_sort();
        vector<int>rnk_new(sz);
        rnk_new[st[0]]=0;
        for(int i=1; i<sz; i++)
        {
            pair<int,int>pre= {rnk[st[i-1]],rnk[(st[i-1]+k)%sz]};
            pair<int,int>cr= {rnk[st[i]],rnk[(st[i]+k)%sz]};
            if(cr==pre)
                rnk_new[st[i]]=rnk_new[st[i-1]];
            else
                rnk_new[st[i]]=rnk_new[st[i-1]]+1;
        }
        rnk=rnk_new;
        k*=2;
    }
}
string getsub(int i,int n)
{
    return s.substr(i,min(n,sz-i));
}
int how_many(string &ss)
{
    int szz=ss.size();
    int lo=0,hi=sz;
    while(lo+1<hi)
    {
        int mid=(lo+hi)>>1;
        string t=getsub(st[mid],szz);
        if(t<=ss)
            lo=mid;
        else
            hi=mid;
    }
    if(getsub(st[lo],szz)!=ss)
        return 0;
    int l=0,r=sz-1;
    while(l<r)
    {
        int mid=(l+r)>>1;
        string t=getsub(st[mid],szz);
        if(t>=ss)
            r=mid;
        else
            l=mid+1;
    }
    return lo-r+1;
}
vector<int> get()
{
    return st;
}
void make_lcp()
{
    int k=0;
    for(int i=0; i<sz-1; i++)
    {
        int pin=rnk[i];
        int j=st[pin-1];
        while(s[i+k]==s[j+k])
            k++;
        lcp[pin]=k;
        k=max(k-1,0);
    }
}
vector<int> get_lcp()
{
    return lcp;
}
};
//String Matching With Bitsets
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 9;
vector<int> v;
bitset<N>bs[26], oc;
int main() {
  int i, j, k, n, q, l, r;
  string s, p;
  cin >> s;
  for(i = 0; s[i]; i++) bs[s[i] - 'a'][i] = 1;
  cin >> q;
  while(q--) {
    cin >> p;
    oc.set();
    for(i = 0; p[i]; i++) oc &= (bs[p[i] - 'a'] >> i);
    cout << oc.count() << endl; // number of occurences
    int ans = N, sz = p.size();
```

```
    int pos = oc._Find_first();
    v.push_back(pos);
    pos = oc._Find_next(pos);
    while(pos < N) {
      v.push_back(pos);
      pos = oc._Find_next(pos);
    }
    for(auto x : v) cout << x << ' '; // position of occurences
    cout << endl;
    v.clear();
    cin >> l >> r; // number of occurences from l to r,where l and r is 1-
indexed
    if(sz > r - l + 1) cout << 0 << endl;
    else cout << (oc >> (l - 1)).count() - (oc >> (r - sz + 1)).count() << endl;
  }
  return 0;
}
//String Factorization
class LyndonFactorization
{
public:
    vector<string> duval(string const &s)
    {
        int n = s.size();
        int i = 0,start=0;
        vector<string> factorization;
        while (i < n)
        {start=i;
            int j = i + 1, k = i;
            while (j < n && s[k] <= s[j])
            {
                if (s[k] < s[j])
                    k = i;
                else
                    k++;
                j++;
            }
            while (i <= k)
            {
                factorization.push_back(s.substr(i, j - k));
                i += j - k;
            }
        }
        return factorization;
    }
};
```

```
//Palindromic Tree
///Must make the string 1 indexed
struct Palindromic_tree
{
    struct node
    {
        int nxt[26];
        int sz;
        int link;
        int tot; /// total number of palindrome ends here
        int fre; /// frequency of current palindrome
        int start;
    };
    string s;
    vector<node>t;
    int id,tt,a; /// tt -> last processed node
    Palindromic_tree(string ss)
    {
        s=ss;
        a=s.size();
        t=vector<node>(a+3);
        t[1].sz=-1,t[1].link=1;
        t[2].sz=0,t[2].link=1;
        tt=id=2;
    }
    int Get_link(int x,int i)
    {
        while(s[i-t[x].sz-1]!=s[i])
            x=t[x].link;
        return x;
    }
    bool Extend(int i)
    {
        tt=Get_link(tt,i);
        int cur=t[tt].link,c=s[i]-'a';
        cur=Get_link(cur,i);

        if(!t[tt].nxt[c]) /// new palindrome
        {
            t[tt].nxt[c]=++id;
            t[id].sz=t[tt].sz+2;
            t[id].link=t[id].sz==1?2:t[cur].nxt[c];
            t[id].tot=1+t[t[id].link].tot;
            tt=t[tt].nxt[c];
            t[tt].fre=1;
            t[tt].start=i-t[tt].sz+1;
            return true;
        }
        tt=t[tt].nxt[c];
        t[tt].fre++;
        return false;
    }
    int how_many()
    {
        int ans=0;
        for(int i=id; i>2; i--)
        {
            t[t[i].link].fre+=t[i].fre;
            ans+=t[i].fre;
        }
        return ans;
    }
```

```
};
//Manacher's Algo
string s;
  cin >> s;
  n = s.size();
  vector<int> d1(n);  // maximum odd length palindrome centered at i
  // here d1[i]=the palindrome has d1[i]-1 right characters from i
  // e.g. for aba, d1[1]=2;
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i);
    while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
      k++;
    }
    d1[i] = k--;
    if (i + k > r) {
      l = i - k;
      r = i + k;
    }
  }
  vector<int> d2(n);  // maximum even length palindrome centered at i
  // here d2[i]=the palindrome has d2[i]-1 right characters from i
  // e.g. for abba, d2[2]=2;
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
      k++;
    }
    d2[i] = k--;
    if (i + k > r) {
      l = i - k - 1;
      r = i + k ;
    }
  }
```

MATH

```
//Matrix
struct Mat
{
    vector<vector<int>>m;
    int x,y;
    Mat(int _x,int _y)
    {
        x=_x,y=_y;
        m.assign(x,vector<int> (y,0));
    }
    inline Mat operator + (const Mat &mt)
    {
        assert(x==mt.x&&y==mt.y);
        Mat ans(x,y);
        for(int i=0; i<x; i++)
        {
            for(int j=0; j<y; j++)
            {
                ans.m[i][j]=(m[i][j]+mt.m[i][j])%mod;
            }
        }
        return ans;
    }
    inline Mat operator - (const Mat &mt)
    {
        assert(x==mt.x&&y==mt.y);
        Mat ans(x,y);
        for(int i=0; i<x; i++)
        {
            for(int j=0; j<y; j++)
            {
                ans.m[i][j]=(m[i][j]-mt.m[i][j])%mod;
            }
        }
        return ans;
    }
    inline Mat operator * (const Mat &mt)
    {
        assert(y==mt.x);
        Mat ans(x,mt.y);
        for(int i = 0; i < x; i++)
        {
            for(int j = 0; j < mt.y; j++)
            {
                for(int k = 0; k < y; k++)
                {
                    ans.m[i][j] = (ans.m[i][j] + 1LL * m[i][k] * mt.m[k][j] %
mod) % mod;
                }
            }
        }
        return ans;
    }
    inline void make_iden()
    {
        assert(x==y);
        for (int i = 0; i < x; i++)
        {
            for (int j = 0; j < x; j++)
                m[i][j] = i == j;
        }
    }
};
```

Dynamic Programming

```
//We Need to Find a increasing subsrquence of
//b numbers such that their difference is minimized
int a,b;
cin>>a>>b;
vi v(a);
cinv(v)
vi dp(a);
for(int k=2; k<=b; k++)
{
```

```
    set<int>s;
    for(int i=a-1; i>=0; i--)
    {
        if(dp[i]==INT_MAX)
        {
            //Not Possible
            continue;
        }
        s.insert(v[i]+dp[i]);//storing last bounds
        auto it=s.upper_bound(v[i]+dp[i]);
        //guranted that we can extend if there exist greater
        if(it==s.end())
        {
            dp[i]=INT_MAX;
        }
        else
        {
            dp[i]=*it-v[i];
            //new difference
        }
    }
}
int ans=*min_element(all(dp));
if(ans==INT_MAX)
    ans=-1;
cout<<ans<<endl;
```

Default Template

```
#include<bits/stdc++.h>
#define ll long long int
#define pb push_back
#define vc vector
#define vi vc<int>
#define vl vc<ll>
#define dbg(x) cerr<<x<<endl;
#define mp(x,y) make_pair(x,y)
#define yes cout<<"YES"<<endl;
#define no cout<<"NO"<<endl;
#define tst int t;cin>>t;while(t--)
#define srt(v) sort(v.begin(),v.end());
#define rsrt(v) sort(v.rbegin(),v.rend());
#define rj ios::sync_with_stdio(false);cin.tie(0);
#define rvs(v) reverse(v.begin(),v.end());
#define F first
#define S second
#define MOD 1000000007
#define gcd(a,b) __gcd(a,b)
#define lcm(a,b) (a*b)/gcd(a,b)
#define PI 2*acos(0.0)
#define pii pair<int,int>
#define fr(i,a,b) for(ll i=a;i<=b;i++)
#define coutv(v) for(auto it:v)cout<<it<<' ';cout<<endl;
#define cinv(v) for(auto &it:v)cin>>it;
#define all(v) v.begin(),v.end()
#define rall(v) v.rbegin(),v.rend()
#define ld long double
#define nl '\n'
const int N=1e5;
using namespace std;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
int my_rand(int l, int r)
{
    return uniform_int_distribution<int>(l, r) (rng);
}
void solve()
{

}
int main()
{
    rj
    //int t;cin>>t;fr(i,1,t) cout<<"Case "<<i<<": ",solve();
    //ll t;scanf("%lld",&t);fr(i,1,t) printf("Case %lld: ",i),solve();
    solve();
    return 0;
}
///--------------Graph Moves----------------------------------------
///const int fx[] = {+1,-1,+0,+0};
///const int fy[] = {+0,+0,+1,-1};
///const int fx[] = {+0,+0,+1,-1,-1,+1,-1,+1}; ///King's move
///const int fy[] = {-1,+1,+0,+0,+1,+1,-1,-1}; ///king's Move
///const int fx[] = {-2,-2,-1,-1,+1,+1,+2,+2}; ///knight's move
///const int fy[] = {-1,+1,-2,+2,-2,+2,-1,+1}; ///knight's move
///-------------------------------------------------------------------
struct custom_hash //For Pair
{
    size_t operator()(const pair<int,int>&x)const
    {
        return ((long long)x.first)^(((long long)x.second)<<32);
    }
};
```

Stress Test

```
@echo off
gen >in
sol <in >out
brute <in >ok
fc out ok
if ErrorLevel 1 exit /b
run
//brute.cpp,sol.cpp,gen.cpp
```

Important formulas

Properties of phi:

1. *If* $gcd(i,n) = d$ *; where* $1 \le i \le n-1$ *then, there are phi* $\left(\frac{n}{d}\right)$ possible values of i.
2. $phi(p^k) = p^k - p^{k-1}$
3. $x^n = x^{\varphi(m)+[n \bmod \varphi(m)]} \bmod m$;where n>= $\log_2 m$

Properties of mod:

1. $ac = bc \pmod m, then \ \ a = b \bmod \frac{m}{g(c,m)}$
2. $mn \pmod n = 0$ *then the smallest number m is equal to* $lcm(n,d)$
3. *if p is prime, then* $(x+y)^p = x^p + y^p \pmod p$.
4. $ab \pmod{ac} = a \ (b \bmod c)$

Properties of Digitsum:

a. $DigitSum(x+y) = DigitSum(DigitSum(x) + DigitSum(y))$
b. $DigitSum(x-y) = DigitSum(DigitSum(x) - DigitSum(y))$
c. $DigitSum(x*y) = DigitSum(DigitSum(x)*y))$
d. $DigitSum(x*y) = DigitSum(x*DigitSum(y))$
e. $DigitSum(x*y) = DigitSum(DigitSum(x)*DigitSum(y))$
f. $DigitSum(x^y) = DigitSum(DigitSum(x)^y)$
g. $DigitSum(x^y) = DigitSum(x^{y\%\varepsilon})$ *where* $DigitSum(x^\varepsilon)$=1

Properties of FLOOR CEIL:

1. $\left\lfloor \frac{n}{m} \right\rfloor = \left\lfloor \frac{n+m-1}{m} \right\rfloor = \left\lfloor \frac{n-1}{m} \right\rfloor + 1$
2. $\left\lfloor \frac{n}{m} \right\rfloor = \left\lceil \frac{n+m-1}{m} \right\rceil = \left\lceil \frac{n-1}{m} \right\rceil + 1$
3. $\sum_{k=1}^{n-1} \left\lfloor \frac{kn}{m} \right\rfloor = \frac{(m-1)(n-1)+gcd(m,n)-1}{2}$

Bit Manipulation Macros

```
#define least_one_pos(x) __builtin_ffs(x)
#define leading_zeros(x) __builtin_clz(x)
#define tailing_zeros(x) __builtin_ctz(x)
#define num_of_one(x) __builtin_popcount(x)
#define msb(x) 32-leading_zeros(x)
```

Important Series and properties

1. $\sum_{n\geq 0} a^n z^n = \frac{1}{1-az}$
2. $\sum_{n\geq 0} \binom{m}{n} z^n = (1+z)^m \quad (m \in \mathbb{Z})$
3. $\sum_{n\geq 0} \binom{m+n-1}{n} z^n = \frac{1}{(1-z)^m} \quad (m \in \mathbb{Z})$
4. $\sum_{n\geq 0} \binom{m+n}{n} z^n = \frac{1}{(1-z)^{m+1}} \quad (m \in \mathbb{Z})$
5. $\sum_{n\geq 0} \binom{n}{m} z^n = \frac{z^m}{(1-z)^{m+1}} \quad (m \in \mathbb{N})$
6. $\sum_{n\geq 0} \binom{p+n}{m} z^n = \frac{z^{m-p}}{(1-z)^{m+1}}$
7. $\sum_{n\geq 0} \frac{z^n}{n!} = e^z$
8. $\sum_{n\geq 0} -1^{n-1} \frac{z^n}{n} = \log(1+z)$
9. Vandermonde $\binom{x+y}{n} = \sum_{k=0}^{n} \binom{x}{k}\binom{y}{n-k}$
10. $\sum_{m=0}^{n} \binom{m}{k} = \sum_{k=0}^{n} \binom{i}{k} = \binom{n+1}{k+1}$
11. $\sum_{k=0}^{n} \binom{n}{k} = 2^n$
12. $\sum_{k=0}^{m} \binom{n+k}{k} = \binom{m+n+1}{m}$
13. $\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$
14. $\sum_{k=1}^{n} k\binom{n}{k} = n2^{n-1}$
15. $a + ar + ar^2 + .. + ar^n = \frac{a(1-r^n)}{1-r}$
16. *sum of xor of all possible subarray* $=$ *(or of all elements* $* \ 2^{n-1}$
17. *sum of sum of all possible subarray* $=$ *(sum of all elements* $* \ 2^{n-1}$
18. De Araangement Recursive Formulla - $D_n = (n-1)(D_{n-1} + D_{n-2})$
19. Catalan Number -

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}, n \geq 0$$

Geometry

```
//Geometry of Mehedi vai
typedef double T;
struct pt {
    T x, y;

    pt operator+(pt p) { return {x + p.x, y + p.y}; }

    pt operator-(pt p) { return {x - p.x, y - p.y}; }

    pt operator*(T d) { return {x * d, y * d}; }

    pt operator/(T d) { return {x / d, y / d}; } // only for floatingpoint
    pt translate(pt v, pt p) { return p + v; }

    pt rot(pt p, double a) {
        return {p.x * cos(a) - p.y * sin(a), p.x * sin(a) + p.y * cos(a)};
    }


};

T sq(pt p) { return p.x * p.x + p.y * p.y; }

double abs(pt p) { return sqrt(sq(p)); }

bool operator==(pt a, pt b) { return a.x == b.x && a.y == b.y; }

bool operator!=(pt a, pt b) { return !(a == b); }

T dot(pt v, pt w) { return v.x * w.x + v.y * w.y; }

T cross(pt v, pt w) { return v.x * w.y - v.y * w.x; }

T orient(pt a, pt b, pt c) { return cross(b - a, c - a); }

//left turn,collinear, right turn = orient(A, B, C) > 0, orient(A, B, C) = 0,
orient(A, B,
pt perp(pt p) { return {-p.y, p.x}; }

double angle(pt v, pt w) {
```

```
    return acos(clamp(dot(v, w) / abs(v) / abs(w), -1.0, 1.0));
}

bool isConvex(vector<pt> p) {
    bool hasPos = false, hasNeg = false;
    for (int i = 0, n = p.size(); i < n; i++) {
        int o = orient(p[i], p[(i + 1) % n], p[(i + 2) % n]);
        if (o > 0) hasPos = true;
        if (o < 0) hasNeg = true;
    }
    return !(hasPos && hasNeg);
}

bool half(pt p) { // true if in blue half
    assert(p.x != 0 || p.y != 0); // the argument of (0,0) isundefined
    return p.y > 0 || (p.y == 0 && p.x < 0);
}

void polarSort(vector<pt> &v) {
    sort(v.begin(), v.end(), [](pt v, pt w) {
        return make_tuple(half(v), 0, sq(v)) <
               make_tuple(half(w), cross(v, w), sq(w));
    });
}

//We can perform a polar sort around some point O other than the
//        origin: we just have to subtract that point O from the vectors #»v and
//#»w when comparing them. This as if we translated the whole plane so
//that O is moved to (0, 0):
//void polarSortAround(p
struct line {
    pt v;
    T c;

// From direction vector v and offset c
    line(pt v, T c) : v(v), c(c) {}

// From equation ax+by=c
    line(T a, T b, T c) : v({b, -a}), c(c) {}

// From points P and Q
    line(pt p, pt q) : v(q - p), c(cross(v, p)) {}

// Will be defined later:
// - these work with T = int
    T side(pt p) { return cross(v, p) - c; }

    double dist(pt p) { return abs(side(p)) / abs(v); }

    line perpThrough(pt p) { return {p, p + perp(v)}; }

    bool cmpProj(pt p, pt q) {
        return dot(v, p) < dot(v, q);
    }

    line translate(pt t) { return {v, c + cross(v, t)}; }

    // - these require T = double

    line shiftLeft(double dist) { return {v, c + dist * abs(v)}; }
//    There is a unique intersection point between two lines l1 and l2 if and
only
//    if # »vl1 × # »vl2 != 0
    bool inter(line l1, line l2, pt &out) {
        T d = cross(l1.v, l2.v);
        if (d == 0) return false;
        out = (l2.v*l1.c - l1.v*l2.c) / d; // requires floating-pointcoordinates
        return true;
    }
    pt proj(pt p) {return p - perp(v)*side(p)/sq(v);}
    pt refl(pt p) {return p - perp(v)*2*side(p)/sq(v);}

};
bool inDisk(pt a, pt b, pt p) {
    return dot(a-p, b-p) <= 0;
}
bool onSegment(pt a, pt b, pt p) {
    return orient(a,b,p) == 0 && inDisk(a,b,p);
}
bool properInter(pt a, pt b, pt c, pt d, pt &out) {
    double oa = orient(c,d,a),
           ob = orient(c,d,b),
           oc = orient(a,b,c),
           od = orient(a,b,d);
// Proper intersection exists iff opposite signs
    if (oa*ob < 0 && oc*od < 0) {
        out = (a*ob - b*oa) / (ob-oa);
        return true;
    }
    return false;
}
//    Segment-point distance
double segPoint(pt a, pt b, pt p) {
    if (a != b) {
        line l(a,b);
        if (l.cmpProj(a,p) && l.cmpProj(p,b)) // if closest toprojection
            return l.dist(p); // output distance toline
    }
    return min(abs(p-a), abs(p-b)); // otherwise distance to A or B
}
// Segment-segment distance
double segSeg(pt a, pt b, pt c, pt d) {
    pt dummy;
    if (properInter(a,b,c,d,dummy))
        return 0;
    return min({segPoint(a,b,c), segPoint(a,b,d),
                segPoint(c,d,a), segPoint(c,d,b)});
}
```

```cpp
double areaPolygon(vector<pt> p) {
    double area = 0.0;
    for (int i = 0, n = p.size(); i < n; i++) {
        area += cross(p[i], p[(i+1)%n]); // wrap back to 0 if i == n-1
    }
    return abs(area) / 2.0;
}
bool above(pt a, pt p) {
    return p.y >= a.y;
}
// check if [PQ] crosses ray from A
bool crossesRay(pt a, pt p, pt q) {
    return (above(a,q) - above(a,p)) * orient(a,p,q) > 0;
}
bool inPolygon(vector<pt> p, pt a, bool strict = true) {
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        if (onSegment(p[i], p[(i+1)%n], a))
            return !strict;
        numCrossings += crossesRay(a, p[i], p[(i+1)%n]);
    }
    return numCrossings & 1; // inside if odd number of crossings
}
```

Triangle

Circumcircle	$r = \frac{abc}{\sqrt{(a+b+c)(b+c-a)(c+a-b)(a+b-c)}}$ $r = \frac{abc}{4 \times AreaOfTriangle}$
Incircle Radius	$\frac{1}{2} \times r(a+b+c) = AreaOfTriangle$
Excircle Radius (If the circle is tangent to side a of the triangle)	$r = IncircleRadius \times \frac{a+b+c}{(b+c-a)}$ $r = 2 \times \frac{AreaOfTriangle}{b+c-a}$
Heron's Formula	$\sqrt{s(s-a)(s-b)(s-c)}$
Sine & Cosine rule	$\frac{a}{sinA} = \frac{b}{sinB} = \frac{c}{sinC} = 2R$ a^2 = b^2 + c^2 − 2bcCosA

Circle

Arc Length	$s = r\theta$ (angle in radian)
Sector Area	area $= \frac{\theta}{2} \times r^2$ (angle in radian)

Chord length	$d = 2 \times r \times \sin\left(\frac{\theta}{2}\right)$ (angle in radian) $d = 2 \times \sqrt{r^2 - x^2}$ (x = Perpendicular Distance from the Centre to Chord)
Outside one another	$C1C2 > r1 + r2$
Touching externally	$C1C2 = r1 + r2$
Intersecting at 2 points	$\lvert r1 + r2 \rvert < C1C2 < r1 + r2$
Touching internally	$C1C2 = \lvert r1 - r2 \rvert$
One inside the other	$C1C2 < \lvert r1 - r2 \rvert$

Others

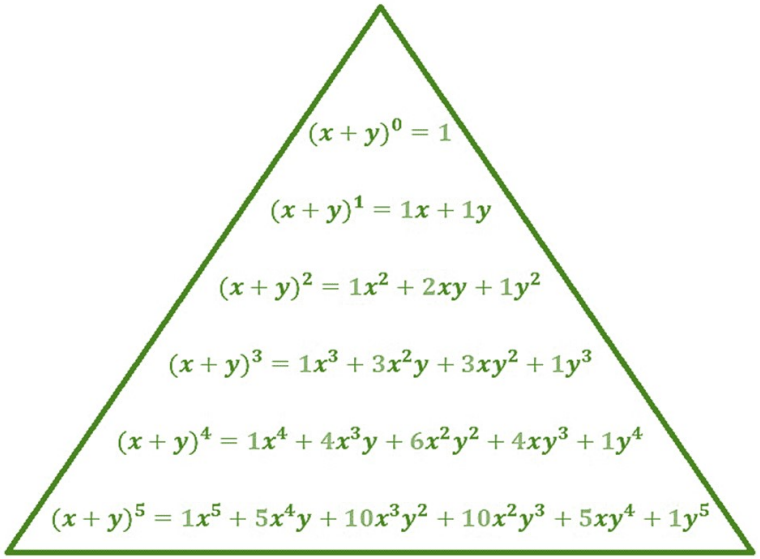
Cube	$area = 6a^2$ $volume = a^3$
Cylinder	$area = 2\pi rh + 2\pi r^2$ $volume = \pi r^2 h$
Cone	$area = \pi rl$ $volume = \frac{1}{3}\pi r^2 h$
sphere	$area = 4\pi r\ 2$ $volume = 4\ 3\ \pi r3$

THEOREM

Burnside's Lemma:

Let G be a finite group that acts on the set X. Let X/G be the set of orbits of X (that is, each element of X/G is an orbit of X). For any element $g \in G$, let X^g be the set of points of X which are fixed by g: $X^g = \{x \in X : g \cdot x = x\}$. Then

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$



Pick's Theorem

Pick's Theorem expresses the area of a polygon, all of whose vertices are lattice points in a coordinate plane, in terms of the number of lattice points inside the polygon and the number of lattice points on the sides of the polygon. The formula is:

$$A = I + \frac{1}{2}B - 1$$

where I is the number of lattice points in the interior and B being the number of lattice points on the boundary.

For the line from (a,b) to (c,d) the number of such points is gcd(c−a,d−b)+1. Especially, if the x and y distances are coprime, only the endpoints are lattice points.